

Project Management

**Università Degli Studi Di
Napoli Federico II
Anno Accademico 20/21
Basi Di Dati I**

**Gruppo 15
Antonio Tagliafierro(N86003148)
Gianpietro Panico(N86003500)**

1 Indice

Sommario

1	Indice	2
2	Introduzione.....	3
	2.1 Analisi del problema	3
3	Progettazione Concettuale	4
	3.1 Class Diagram	4
	3.2 Ristrutturazione del Class Diagram	5
	3.2.1 Rimozione degli attributi multipli	5
	3.2.2 Rimozione delle gerarchie di specializzazione	5
	3.2.3 Rimozione degli attributi strutturati	5
	3.3 Dizionario delle classi	6
	3.4 Dizionario delle associazioni	8
	3.5 Dizionario dei vincoli	9
4	Progettazione Logica	10
	4.1 Schema logico	10
5	Progettazione fisica	11
	5.1 Definizione tabelle	11
	5.2 Viste	16
	5.2.1 Dipendenti_De_i_MeetingF	16
	5.2.2 Dipendenti_De_i_MeetingT	16
	5.2.3 Dipendenti_De_i_Progetti	16
	5.2.4 Progetti_Per_Tipologia	17
	5.3 Procedure	17
	5.3.1 Attiva_Dipendente	17
	5.3.2 Disattiva_Dipendente	18
	5.3.3 Fine_Progetto	18
	5.4 Sequenze	19
	5.5 Trigger e Funzioni	20
6	Popolamento con Dati d'Esempio	38

2 Introduzione

La seguente documentazione provvede ad analizzare il database per l'applicativo Java sviluppato dagli studenti Gianpietro Panico e Antonio Tagliaferro, del Corso di Laurea in Informatica dell'Università degli Studi di Napoli Federico II.

2.1 Analisi del problema

La richiesta è di sviluppare un sistema informativo, composto da una base di dati relazionale e un applicativo Java dotato di GUI, per la gestione di progetti in un'azienda, tenendo traccia dei partecipanti ai progetti, identificando i ruoli per ognuno di essi (richiedendo uno ed un solo project manager per ogni progetto).

Ad ogni progetto è associata una tipologia ed uno o più ambiti.

L'applicativo dovrà permettere anche l'organizzazione di meeting fisici in sale riunioni, o telematici su una piattaforma di videoconferenza.

Inoltre, è richiesto il salvataggio delle partecipazioni ai progetti ed ai meeting, ai fini della valutazione del singolo partecipante.

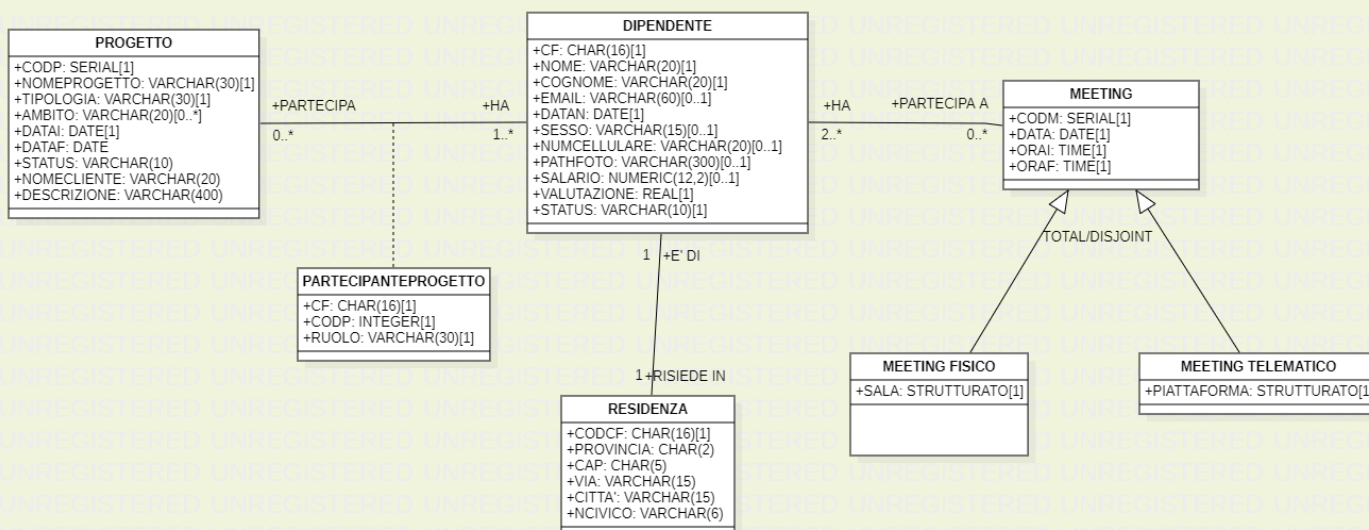
In fase di creazione di un nuovo progetto, è richiesta la ricerca dei dipendenti, da aggiungere al progetto, mediante criteri quali: salario, valutazione e tipologia dei progetti ai quali hanno preso già parte in passato.

3 Progettazione Concettuale

In questo capitolo inizia la progettazione della base di dati al livello di astrazione più alto. Dal risultato dell'analisi dei requisiti che devono essere soddisfatti si arriverà ad uno schema concettuale indipendente dalla struttura dei dati e dall'implementazione fisica: in tale schema concettuale, che verrà rappresentato usando un Class Diagram UML, si evidenzieranno le entità (concetti) rilevanti ai fini della rappresentazione dei dati e le relazioni che intercorrono tra esse; si delineeranno anche eventuali vincoli da imporre.

3.1 Class Diagram

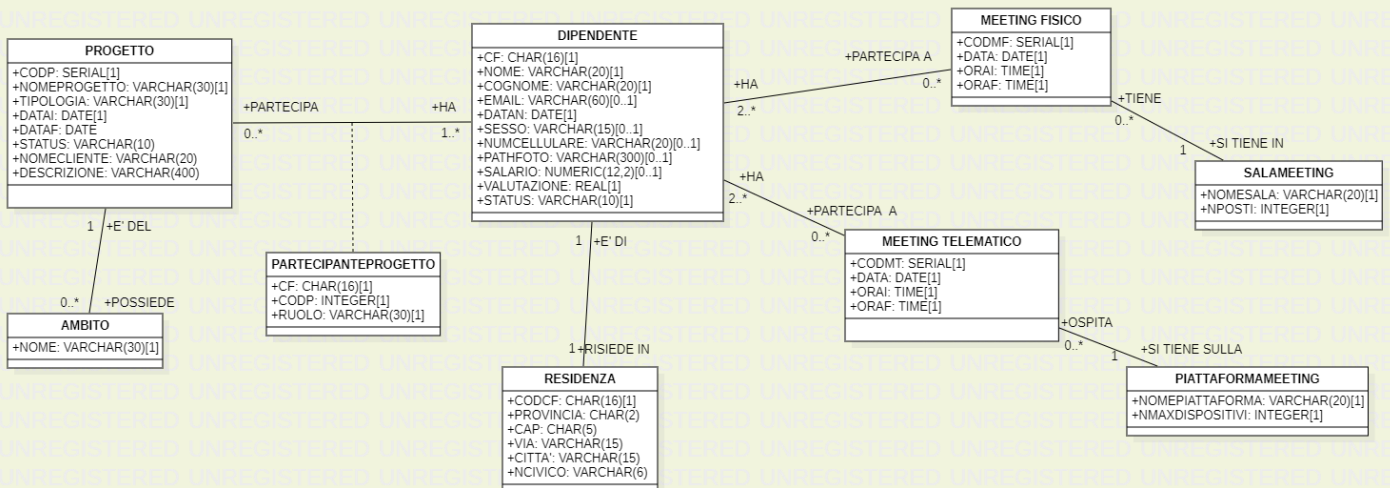
Segue L'immagine del Class Diagram della base di dati.



3.2 Ristrutturazione del Class Diagram

Al fine di rendere il class diagram idoneo alla traduzione in schemi relazionali e di migliorare l'efficienza dell'implementazione si procede alla ristrutturazione dello stesso.

Segue l'immagine del Class Diagram ristrutturato.



3.2.1 Rimozione degli attributi multipli

Abbiamo provveduto alla rimozione dell'attributo multiplo "Ambito" in "Progetto" creando una nuova classe "Ambito"

3.2.2 Rimozione delle gerarchie di specializzazione

Inoltre, per ristrutturare il class diagram, era necessario trasformare la specializzazione dell'entità "Meeting"; Abbiamo eliminato la specializzazione optando per lo schiacciamento della classe generale "Meeting" nelle classi specializzate "SalaMeeting" e "Piattaforma".

3.2.3 Rimozione degli attributi strutturati

Abbiamo riscontrato la presenza di due attributi strutturati in "Meeting": "Sala" e "Piattaforma";

Per cui abbiamo provveduto alla loro eliminazione come attributi andando a creare due nuove classi "Salameeting" e "Piattaforma".

3.3 Dizionario delle classi

Classi	Descrizione	Attributi
<i>Dipendente</i>	<i>Descrittore dei dipendenti dell'azienda</i>	CF (Char(16)): codice fiscale del dipendente. NOME (Varchar(20)): nome del dipendente. COGNOME (Varchar(20)): cognome del dipendente. EMAIL (Varchar(60)): E-mail del dipendente. DATAN (Date): data di nascita del dipendente. SESSO (Varchar(15)): sesso del dipendente. NUMCELLULARE (Varchar(20)): numero di cellulare del dipendente. PATHFOTO (Varchar(300)): path della foto del dipendente. SALARIO (Numeric(12,2)): salario del dipendente. VALUTAZIONE (Real): valutazione del dipendente. STATUS (Varchar(10)): status del dipendente.
<i>Residenza</i>	<i>Descrittore della residenza dei dipendenti</i>	CODCF (Char(16)): codice fiscale del residente. PROVINCIA (Char(2)): provincia del residente. CAP (Char(2)): codice postale del residente. VIA (Varchar(15)): via del residente CITTA' (Varchar(15)): città del residente. NCIVICO (Varchar(6)): numero civico del residente.
<i>Progetto</i>	<i>Descrittore dei progetti dell'aziende</i>	CODP (Serial): codice identificativo del progetto. NOMEPROGETTO (Varchar(30)): nome del progetto. TIPOLOGIA (Varchar(10)): tipolozia del progetto. DATAI (Date): data iniziale del progetto. DATAF (Date): data di scadenza del progetto. STATUS (Varchar(10)): status del progetto. NOMECLIENTE (Varchar(20)): nome del cliente del progetto; DESCRIZIONE (Varchar(400)): descrizione del progetto.

<i>Ambito</i>	<i>Descrittore degli ambiti dei progetti</i>	NOME (Varchar(30)): ambito del progetto associato.
<i>Partecipante Progetto</i>	<i>Descrittore della partecipazione di un dipendente ad un progetto</i>	CF (Char(16)): codice fiscale del partecipante al progetto. CODP (Integer): codice del progetto a cui partecipa il dipendente. RUOLO (Varchar(30)): ruolo con cui il dipendente partecipa al progetto.
<i>MeetingFisico</i>	<i>Descrittore dei meeting dell'azienda tenuti in presenza</i>	CODMF (Serial): codice identificativo del meeting fisico. DATA (Date): data in cui si terrà il meeting. ORAI (Time): ora di inizio del meeting. ORAF (Time): ora finale del meeting.
<i>Meeting Telematico</i>	<i>Descrittore dei meeting dell'azienda tenuti in modo telematico</i>	CODMT (Serial): codice identificativo del meeting telematico. DATA (Date): data in cui si terrà il meeting. ORAI (Time): ora di inizio del meeting. ORAF (Time): ora finale del meeting.
<i>SalaMeeting</i>	<i>Descrittore della sala che ospita un meeting fisico</i>	CF (Char(16)): codice fiscale del partecipante al progetto. CODP (Integer): codice del progetto a cui partecipa il dipendente. RUOLO (Varchar(30)): ruolo con cui il dipendente partecipa al progetto.
<i>Piattaforma Meeting</i>	<i>Descrittore della piattaforma che ospita un meeting telematico</i>	NOMEPIATTAFORMA (Varchar(20)): nome della piattaforma in cui si terrà il meeting telematico. NMAXDISPOSITIVI (Integer): numero massimo dei dispositivi collegabili al meeting.

3.4 Dizionario delle associazioni

Nome	Descrizione	Classi Coinvolte
<i>Abitazione</i>	<i>Esprime l'appartenenza di una residenza ad un dipendente.</i>	Dipendente [1] ruolo risiede in: indica il dipendente che risiede in una residenza Residenza [1] ruolo è di: indica la residenza dove risiede il dipendente
<i>Partecipanteprogetto</i>	<i>Esprime la partecipazione dei dipendenti ai progetti.</i>	Progetto [1..*] ruolo ha: esprime i dipendenti che partecipano ad un progetto. Dipendente [0..*] ruolo partecipa: esprime per ogni dipendenti i progetti a cui partecipa.
<i>Ha_ambito</i>	<i>Esprime l'appartenenza di un ambito ad un progetto</i>	Progetto [0..*] ruolo possiede: indica gli ambiti del progetto. Ambito [1] ruolo è del: indica l'ambito del progetto.
<i>PartecipanteMeetingFisico</i>	<i>Esprime la partecipazione dei dipendenti ai meeting fisici</i>	Dipendente [0..*] ruolo partecipa a: indica i meeting fisici a cui partecipa il dipendente. Meeting Fisico [2..*] ruolo ha: indica i dipendenti che fanno parte del meeting.
<i>PartecipanteMeetingTelematico</i>	<i>Esprime la partecipazione dei dipendenti ai meeting telematici</i>	MeetingTelematico [2..*] ruolo ha: indica i dipendenti che fanno parte del meeting. Dipendente [0..*] ruolo partecipa a: indica i meeting fisici a cui partecipa il dipendente.
<i>OspitaFisico</i>	<i>Esprime la sala in cui si tiene il meeting</i>	MeetingFisico [1] ruolo si tiene in: indica la sala in cui si tiene il meeting. SalaMeeting [0..*] ruolo tiene: indica i meeting che si tengono nella sala.

<i>OspitaTelematico</i>	<i>Esprime la piattaforma in cui si tiene il meeting</i>	MeetingTelematico [1] ruolo si tiene sulla : indica la piattaforma su cui si tiene il meeting. SalaMeeting [o..*] ruolo ospita : indica i meeting che si tengono sulla piattaforma.
-------------------------	--	--

3.5 Dizionario dei vincoli

Vincoli di chiave primaria ed esterna omessi per brevità.

Nome Vincolo	Descrizione
ControlloStatus	Controlla che lo status del dipendente sia o 'ATTIVO' o 'NON ATTIVO'
ControlloCF	Controlla che il codice fiscale sia di forma legittima, ovvero devono essere composte da: sei lettere, due numeri, una lettera, 2 numeri, una lettera, tre numeri, una lettera.
ControlloCognome	Controlla che il cognome sia composto da sole lettere.
ControlloNome	Controlla che il nome del dipendente sia composto da sole lettere.
Dipendente_email_key	Controlla che non ci siano più mail uguali.
UnicitàPartecipanteF	Controlla che un meeting fisico non abbia duplicati tra i partecipanti.
UnicitàPartecipanteT	Controlla che un meeting telematico non abbia duplicati tra i partecipanti.
UnicoRuolo	Controlla che in un progetto, un dipendente abbia un solo ruolo.

4 Progettazione Logica

In questo capitolo sarà trattata la fase successiva della progettazione della base di dati. Si tradurrà lo schema concettuale in uno schema logico. Negli schemi relazionali che seguiranno le chiavi primarie sono indicate con una singola sottolineatura mentre le chiavi esterne con una doppia sottolineatura.

4.1 Schema logico

Dipendente (CF, Nome, Cognome, Email, DataN, Sesso, NumCellulare, PathFoto, Salario, Valutazione, Status)

Progetto (CodP, NomeProgetto, Tipologia, DataI, DataF, Status, NomeCliente, Descrizione)

PartecipanteProgetto (CF, CodP, Ruolo)

CF->Dipendente.CF, CodP->Progetto.CodP;

Ambito (Nome, CodP)

CodP->Progetto.CodP;

Residenza (CodF, Provincia, CAP, Via, Città, NCivico)

CodF->Dipendente.CF;

MeetingFisico (CodMF, Data, OraI, OraF, NomeSala)

NomeSala->Sala.NomeSala;

PartecipanteMeetingFisico (CodF, CodMF)

CodF->Dipendente.CF, CodMF->MeetingFisico.CodMF;

MeetingTelematico (CodMT, Data, OraI, OraF, Nome Piattaforma)

NomePiattaforma->Piattaforma.NomePiattaforma;

PartecipanteMeetingTelematico (CodF, CodMT)

CodF->Dipendente.CF, CodMT->MeetingTelematico.CodMT;

SalaMeeting (NomeSala, NPosti)

PiattaformaMeeting (NomePiattaforma, NMaxDispositivi)

5 Progettazione fisica

Per realizzare la base di dati è stato utilizzato il DBMS Postgresql.

5.1 Definizione tabelle

Seguono le definizioni delle tabelle estratte dallo script di creazione del database.

```
/**
 * TABELLA: AMBITO
 * Crea la tabella e implementa i vincoli più semplici.
 */
-- Crea la tabella AMBITO e Vincolo di Chiave Esterna

CREATE TABLE PUBLIC.AMBITO
(
    NOME CHARACTER VARYING (30) NOT NULL,
    CODP INTEGER,
    CONSTRAINT FK1_AMBITO FOREIGN KEY (CODP)
        REFERENCES PUBLIC.PROGETTO (CODP)
        ON UPDATE NO ACTION
        ON DELETE CASCADE
)

/**
 * TABELLA: Dipendente
 * Crea la tabella e implementa i vincoli.
 */
--Crea la tabella Dipendente ,vincolo di chiave primaria, Controllo Status, Dipendente_Email_Key,
ControlloCf, Controllo cognome, Controllo nome.

CREATE TABLE PUBLIC.DIPENDENTE
(
    CODF CHARACTER (16) NOT NULL,
    NOME CHARACTER VARYING (20) NOT NULL,
    COGNOME CHARACTER VARYING (20) NOT NULL,
    EMAIL CHARACTER VARYING (60),
    DATAN DATE NOT NULL,
    SESSO CHARACTER VARYING (15),
    NUMCELLULARE CHARACTER VARYING (20),
    PATHFOTO CHARACTER VARYING (300),
    SALARIO NUMERIC (12,2),
    VALUTAZIONE REAL NOT NULL DEFAULT 0,
    STATUS CHARACTER VARYING (10) NOT NULL DEFAULT 'ATTIVO'::CHARACTER VARYING,
    CONSTRAINT DIPENDENTE_PKEY PRIMARY KEY (CODF),
    CONSTRAINT DIPENDENTE_EMAIL_KEY UNIQUE (EMAIL),
    CONSTRAINT "CONTROLLO STATUS" CHECK (STATUS::TEXT = ANY
(ARRAY['ATTIVO'::CHARACTER VARYING::TEXT, 'NON ATTIVO'::CHARACTER
VARYING::TEXT])),
```

```

CONSTRAINT CONTROLLOC CHECK (CODF ~* '^[A-ZA-Z]{6}[0-9]{2}[A-ZA-Z][0-9]{2}[A-
ZA-Z][0-9]{3}[A-ZA-Z]$'::TEXT),
CONSTRAINT CONTROLLOCOGNOME CHECK (COGNOME::TEXT ~* '^[A-ZA-Z ]+$'::TEXT),
CONSTRAINT CONTROLLOMOME CHECK (NOME::TEXT ~* '^[A-ZA-Z ]+$'::TEXT)
)

```

```

/**
 * TABELLA: MeetingFisico
 * Crea la tabella e implementa i vincoli più semplici.
 */
--Crea la tabella MeetingFisico, vincolo di chiave primaria e vincolo di chiave esterna

```

```

CREATE TABLE PUBLIC.MEETINGFISICO
(
    CODMF INTEGER NOT NULL DEFAULT
NEXTVAL('MEETINGFISICO_CODMF_SEQ'::REGCLASS),
    DATAMF DATE NOT NULL,
    ORAI TIME WITHOUT TIME ZONE NOT NULL,
    ORAF TIME WITHOUT TIME ZONE NOT NULL,
    NOMESALA CHARACTER VARYING (20),
    CONSTRAINT MEETINGFISICO_PKEY PRIMARY KEY (CODMF),
    CONSTRAINT FK_SALAMEETING FOREIGN KEY (NOMESALA)
    REFERENCES PUBLIC.SALAMEETING (NOMESALA)
    ON UPDATE CASCADE
    ON DELETE NO ACTION
)

```

```

/**
 * TABELLA: MeetingTelematico
 * Crea la tabella e implementa i vincoli più semplici.
 */
--Crea la tabella MeetingTelematico, vincolo di chiave primaria e vincolo di chiave esterna

```

```

CREATE TABLE PUBLIC.MEETINGTELEMATICO
(
    CODMT INTEGER NOT NULL DEFAULT
NEXTVAL('MEETINGTELEMATICO_CODMT_SEQ'::REGCLASS),
    DATAMT DATE NOT NULL,
    ORAI TIME WITHOUT TIME ZONE NOT NULL,
    ORAF TIME WITHOUT TIME ZONE NOT NULL,
    NOMEPIATTAFORMA CHARACTER VARYING (20) NOT NULL,
    CONSTRAINT MEETINGTELEMATICO_PKEY PRIMARY KEY (CODMT),
    CONSTRAINT FK_PIATTAFORMAMEETING FOREIGN KEY (NOMEPIATTAFORMA)
    REFERENCES PUBLIC.PIATTAFORMAMEETING (NOMEPIATTAFORMA)
    ON UPDATE CASCADE
    ON DELETE NO ACTION
)

```

```

/**
 * TABELLA: PartecipanteMeetingFisico
 * Crea la tabella e implementa i vincoli più semplici.
 */
--Crea la tabella PartecipanteMeetingFisico, UnicitàPartecipanteF e vincoli di chiave esterna.

```

```

CREATE TABLE PUBLIC.PARTECIPANTEMEETINGFISICO
(
  CODF CHARACTER (16),
  CODMF INTEGER,
  CONSTRAINT "UNICITÀPARTECIPANTEF" UNIQUE (CODF, CODMF),
  CONSTRAINT FK1_PMF FOREIGN KEY (CODF)
    REFERENCES PUBLIC.DIPENDENTE (CODF)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  CONSTRAINT FK2_PMF FOREIGN KEY (CODMF)
    REFERENCES PUBLIC.MEETINGFISICO (CODMF)
    ON UPDATE CASCADE
    ON DELETE CASCADE
)

```

```

/**
 * TABELLA: PartecipanteMeetingTelematico
 * Crea la tabella e implementa i vincoli più semplici.
 */
--Crea la tabella PartecipanteMeetingTelematico, UnicitàPartecipanteT e vincoli di chiave esterna.

```

```

CREATE TABLE PUBLIC.PARTECIPANTEMEETINGTELEMATICO
(
  CODF CHARACTER (16),
  CODMT INTEGER,
  CONSTRAINT "UNICITÀPARTECIPANTET" UNIQUE (CODF, CODMT),
  CONSTRAINT FK1_MT FOREIGN KEY (CODF)
    REFERENCES PUBLIC.DIPENDENTE (CODF)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  CONSTRAINT FK2_MT FOREIGN KEY (CODMT)
    REFERENCES PUBLIC.MEETINGTELEMATICO (CODMT)
    ON UPDATE CASCADE
    ON DELETE CASCADE
)

```

```

/**
 * TABELLA: PartecipanteProgetto
 * Crea la tabella e implementa i vincoli più semplici.
 */
--Crea la tabella PartecipanteProgetto, UnicoRuolo e vincoli di chiave esterna.

```

```

CREATE TABLE PUBLIC.PARTECIPANTEPROGETTO
(
  CODF CHARACTER (16),
  CODP INTEGER,
  RUOLO CHARACTER VARYING (30) NOT NULL,
  CONSTRAINT UNICORUOLO UNIQUE (CODF, CODP),
  CONSTRAINT FK1_PARPRO FOREIGN KEY (CODF)
    REFERENCES PUBLIC.DIPENDENTE (CODF)
    ON UPDATE CASCADE
    ON DELETE NO ACTION,
  CONSTRAINT FK2_PARPRO FOREIGN KEY (CODP)
    REFERENCES PUBLIC.PROGETTO (CODP)
    ON UPDATE CASCADE
    ON DELETE CASCADE
)

```

```

/**
 * TABELLA: PiattaformaMeeting
 * Crea la tabella e implementa i vincoli più semplici.
 */
--Crea la tabella PiattaformaMeeting e vincolo di chiave primaria.

```

```

CREATE TABLE PUBLIC.PIATTAFORMAMEETING
(
  NOMEPIATTAFORMA CHARACTER VARYING (20) NOT NULL,
  NMAXDISPOSITIVI INTEGER NOT NULL,
  CONSTRAINT PIATTAFORMAMEETING_PKEY PRIMARY KEY (NOMEPIATTAFORMA)
)

```

```

/**
 * TABELLA: Progetto
 * Crea la tabella e implementa i vincoli più semplici.
 */
--Crea la tabella Progetto e vincolo di chiave primaria.

CREATE TABLE PUBLIC.PROGETTO
(
  CODP INTEGER NOT NULL DEFAULT NEXTVAL('PROGETTO_CODP_SEQ'::REGCLASS),
  NOMEPROGETTO CHARACTER VARYING (30) NOT NULL,
  TIPOLOGIA CHARACTER VARYING (30) NOT NULL,
  DATAI DATE NOT NULL,
  DATAF DATE,
  NOMECLIENTE CHARACTER VARYING (20)
  DESCRIZIONE CHARACTER VARYING (400)
  STATUS CHARACTER VARYING (10) DEFAULT 'IN CORSO'::CHARACTER VARYING,
  CONSTRAINT PROGETTO_PKEY PRIMARY KEY (CODP)
)

```

```

/**
 * TABELLA: Residenza
 * Crea la tabella e implementa i vincoli più semplici.
 */
--Crea la tabella Residenza e vincolo di chiave esterna.

```

```

CREATE TABLE PUBLIC.RESIDENZA
(
  CODF CHARACTER (16),
  PROVINCIA CHARACTER (2),
  CAP CHARACTER (5),
  VIA CHARACTER VARYING (15),
  NCIVICO CHARACTER VARYING (6),
  "CITTÀ" CHARACTER VARYING (15),
  CONSTRAINT FK_RESIDENZA FOREIGN KEY (CODF)
    REFERENCES PUBLIC.DIPENDENTE (CODF)
    ON UPDATE CASCADE
    ON DELETE CASCADE
)

```

```

/**
 * TABELLA: SalaMeeting
 * Crea la tabella e implementa i vincoli più semplici.
 */
--Crea la tabella SalaMeeting e vincolo di chiave primaria.

```

```

CREATE TABLE PUBLIC.SALAMEETING
(
  NOMESALA CHARACTER VARYING (20) NOT NULL,
  NPOSTI INTEGER NOT NULL,
  CONSTRAINT SALAMEETING_PKEY PRIMARY KEY (NOMESALA)
)

```


5.2 Viste

5.2.1 Dipendenti_De MeetingF

La vista mostra per ogni meeting fisico i dipendenti che ne prendono parte.

```
CREATE OR REPLACE VIEW PUBLIC.DIPENDENTI_DEI_MEETINGF
AS
SELECT MEETINGFISICO.CODMF,
       MEETINGFISICO.DATAMF,
       MEETINGFISICO.NOMESALA,
       PARTECIPANTEMEETINGFISICO.CODF
FROM MEETINGFISICO
JOIN PARTECIPANTEMEETINGFISICO USING (CODMF);
```

5.2.2 Dipendenti_De MeetingT

La vista mostra per ogni meeting telematico i dipendenti che ne prendono parte.

```
CREATE OR REPLACE VIEW PUBLIC.DIPENDENTI_DEI_MEETINGT
AS
SELECT MEETINGTELEMATICO.CODMT,
       MEETINGTELEMATICO.DATAMT,
       MEETINGTELEMATICO.NOMEPIATTAFORMA,
       PARTECIPANTEMEETINGTELEMATICO.CODF
FROM MEETINGTELEMATICO
JOIN PARTECIPANTEMEETINGTELEMATICO USING (CODMT);
```

5.2.3 Dipendenti_De Progetti

La vista mostra per ogni progetto i dipendenti che ne prendono parte.

```
CREATE OR REPLACE VIEW PUBLIC.DIPENDENTI_DEI_PROGETTI
AS
SELECT PROGETTO.NOMEPROGETTO,
       PROGETTO.TIPOLOGIA,
       PARTECIPANTEPROGETTO.RUOLO,
       PARTECIPANTEPROGETTO.CODF
FROM PROGETTO
JOIN PARTECIPANTEPROGETTO USING (CODP);
```

5.2.4 Progetti_Per_Tipologia

La vista mostra i dipendenti che prendono parte ai progetti divisi per tipologia.

```
CREATE OR REPLACE VIEW PUBLIC.PROGETTI_PER_TIPOLOGIA
AS
SELECT PROGETTO.CODP,
       SQ.CODF,
       PROGETTO.TIPOLOGIA,
       SQ.NOME,
       SQ.COGNOME,
       SQ.VALUTAZIONE,
       SQ.SALARIO
FROM PROGETTO
JOIN ( SELECT PARTECIPANTEPROGETTO.CODP,
        PARTECIPANTEPROGETTO.CODF,
        DIPENDENTE.NOME,
        DIPENDENTE.COGNOME,
        DIPENDENTE.VALUTAZIONE,
        DIPENDENTE.SALARIO
      FROM PARTECIPANTEPROGETTO
      JOIN DIPENDENTE USING (CODF)) SQ USING (CODP);
```

5.3 Procedure

5.3.1 Attiva_Dipendente

La procedura cambia lo status del dipendente scelto in ATTIVO.

```
CREATE OR REPLACE PROCEDURE public.attiva_dipendente(
    codicef character,
    nuovosalario numeric)
LANGUAGE 'plpgsql'
AS $BODY$
BEGIN

    UPDATE DIPENDENTE
    SET status = 'ATTIVO',salario=nuovosalario
    WHERE codF = codiceF;

END;
$BODY$;
```

5.3.2 Disattiva_Dipendente

La procedura cambia lo status del dipendente scelto in NON ATTIVO.

```
CREATE OR REPLACE PROCEDURE public.disattiva_dipendente(  
    codicef character)  
LANGUAGE 'plpgsql'  
AS $BODY$  
BEGIN  
  
    UPDATE DIPENDENTE  
    SET status = 'NON ATTIVO' ,salario = 0  
    WHERE codF = codiceF;  
  
END;  
$BODY$;
```

5.3.3 Fine_Progetto

La procedura cambia lo status del progetto scelto in FINITO.

```
CREATE OR REPLACE PROCEDURE public.fine_progetto(  
    codicep integer)  
LANGUAGE 'plpgsql'  
AS $BODY$  
BEGIN  
  
    UPDATE PROGETTO  
    SET status = 'FINITO'  
    WHERE codP = codiceP;  
  
END;  
$BODY$;
```

5.4 Sequenze

Le seguenti sequenze sono state create in automatico in seguito all'inserimento degli attributi "CodP", "CodMF", "CodMT" come tipo Serial.

```
CREATE SEQUENCE public.meetingfisico_codmf_seq  
  INCREMENT 1  
  START 1  
  MINVALUE 1  
  MAXVALUE 2147483647  
  CACHE 1;
```

```
CREATE SEQUENCE public.meetingtelematico_codmt_seq  
  INCREMENT 1  
  START 1  
  MINVALUE 1  
  MAXVALUE 2147483647  
  CACHE 1;
```

```
CREATE SEQUENCE public.progetto_codp_seq  
  INCREMENT 1  
  START 1  
  MINVALUE 1  
  MAXVALUE 2147483647  
  CACHE 1;
```

5.5 Trigger e Funzioni

Trigger che richiama la funzione per controllare che una sala non sia già occupata prima dell'inserimento di un nuovo meeting Fisico.

```
CREATE FUNCTION public."Controllo Sovrapposizione Meetinginuna Sala"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
BEGIN
  IF EXISTS (SELECT *
             FROM MEETINGFISICO AS MF
             WHERE NEW.nomeSala = MF.nomeSala AND NEW.CODMF <> MF.CODMF AND
NEW.dataMF = MF.dataMF AND
             ((NEW.OraI BETWEEN MF.OraI AND MF.OraF) OR
              (NEW.OraF BETWEEN MF.OraI AND MF.OraF) OR (MF.OraI BETWEEN NEW.OraI
AND NEW.OraF)
              OR (MF.OraF BETWEEN NEW.OraI AND NEW.OraF))) THEN
    RAISE EXCEPTION 'ATTENZIONE sala già occupata';
  END IF;

  RETURN NEW;
END;
$BODY$;

CREATE TRIGGER "Trigger_controllosovrapposizionemeetinginunasala"
  BEFORE INSERT
  ON public.meetingfisico
  FOR EACH ROW
  EXECUTE PROCEDURE public."Controllo Sovrapposizione Meetinginuna Sala"();
```

Trigger che richiama la funzione per controllare che una sala non sia già occupata prima della modifica di un meeting Fisico.

```
CREATE FUNCTION public."Controllo Sovrapposizione Meetinginuna Sala"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
BEGIN
  IF EXISTS (SELECT *
             FROM MEETINGFISICO AS MF
             WHERE NEW.nomeSala = MF.nomeSala AND NEW.CODMF<>MF.CODMF AND
NEW.dataMF = MF.dataMF AND
             ((NEW.OraI BETWEEN MF.OraI AND MF.OraF) OR
              (NEW.OraF BETWEEN MF.OraI AND MF.OraF)OR(MF.OraI BETWEEN NEW.OraI
AND NEW.OraF)
              OR(MF.OraF BETWEEN NEW.OraI AND NEW.OraF))) THEN
    RAISE EXCEPTION 'ATTENZIONE sala già occupata';
  END IF;

  RETURN NEW;
END;
$BODY$;

CREATE TRIGGER controllo_sovrapposizione_saladoupd
  BEFORE UPDATE
  ON public.meetingfisico
  FOR EACH ROW
  EXECUTE PROCEDURE public."Controllo Sovrapposizione Meetinginuna Sala"();
```

Trigger che richiama la funzione per controllare che un partecipante non sia già occupato nell'arco temporale del meeting in cui è stato aggiunto prima dell'inserimento di un nuovo meeting Fisico.

```
CREATE FUNCTION public."Controllo Sovrapposizione Meeting Per PartecipanteMeetingFisico"()
    RETURNS trigger
    LANGUAGE 'plpgsql'
    COST 100
    VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
    dataMeeting DATE;
    oraInizioMeeting TIME;
    oraFineMeeting TIME;
BEGIN
    SELECT MF.Datamf, MF.OraI, MF.OraF INTO dataMeeting, oraInizioMeeting, oraFineMeeting
    FROM PARTECIPANTEMEETINGFISICO AS PMF NATURAL JOIN MEETINGFISICO AS MF
    WHERE NEW.CodF = PMF.CodF AND NEW.CodMF = MF.CodMF;

    IF EXISTS (SELECT *
                FROM PARTECIPANTEMEETINGFISICO AS PMF NATURAL JOIN
                MEETINGFISICO AS MF
                WHERE NEW.CodF = PMF.CodF AND dataMeeting = MF.Datamf AND
                ((oraInizioMeeting BETWEEN MF.OraI AND MF.OraF) OR
                 (oraFineMeeting BETWEEN MF.OraI AND MF.OraF)OR(MF.OraI BETWEEN
oraInizioMeeting AND oraFineMeeting)
                OR(MF.OraF BETWEEN oraInizioMeeting AND oraFineMeeting)))
        OR EXISTS (SELECT *
                    FROM PARTECIPANTEMEETINGTELEMATICO AS PMT NATURAL JOIN
                    MEETINGTELEMATICO AS MT
                    WHERE NEW.CodF = PMT.CodF AND dataMeeting = MT.Datamt AND
                    ((oraInizioMeeting BETWEEN MT.OraI AND MT.OraF) OR
                     (oraFineMeeting BETWEEN MT.OraI AND MT.OraF)OR(MT.OraI
BETWEEN oraInizioMeeting AND oraFineMeeting)
                    OR(MT.OraF BETWEEN oraInizioMeeting AND oraFineMeeting)))
    THEN
        RAISE EXCEPTION 'In quelle ore questo dipendente partecipa già ad un altro meeting';
    END IF;

    RETURN NEW;
END;
$BODY$;

CREATE TRIGGER "Trigger_controllosovrapposizionemeetingperpartecipantemeetingfi"
    BEFORE INSERT
    ON public.partecipantemeetingfisico
    FOR EACH ROW
    EXECUTE PROCEDURE public."Controllo Sovrapposizione Meeting Per
PartecipanteMeetingFisico"();
```


Trigger che richiama la funzione che decrementa la valutazione di un partecipante al meeting Fisico dopo la sua eliminazione.

```
CREATE FUNCTION public."Decrementa Valutazione Partecipanti Meeting Eliminati"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
  valutazioneAggiornata REAL;
BEGIN
  SELECT valutazione INTO valutazioneAggiornata
  FROM DIPENDENTE
  WHERE CodF = OLD.CodF;

  valutazioneAggiornata := valutazioneAggiornata - 0.5;

  UPDATE DIPENDENTE
  SET valutazione = valutazioneAggiornata
  WHERE CodF = OLD.CodF;

  RETURN NULL;
END;
$BODY$;

CREATE TRIGGER "Trigger_decrementavalutazionepartecipantemeetingfisico"
  AFTER DELETE
  ON public.partecipantemeetingfisico
  FOR EACH ROW
  EXECUTE PROCEDURE public."Decrementa Valutazione Partecipanti Meeting Eliminati"();
```

Trigger che richiama la funzione che incrementa la valutazione di un partecipante al meeting dopo essere stato aggiunto ad un meeting Fisico.

```
CREATE FUNCTION public."Incrementa Valutazione Partecipanti Meeting"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
  valutazioneAggiornata REAL;
BEGIN
  SELECT valutazione INTO valutazioneAggiornata
  FROM DIPENDENTE
  WHERE CodF = NEW.CodF;

  valutazioneAggiornata := valutazioneAggiornata + 0.5;

  UPDATE DIPENDENTE
  SET valutazione = valutazioneAggiornata
  WHERE CodF = NEW.CodF;

  RETURN NULL;
END;
$BODY$;

CREATE TRIGGER "Trigger_incrementovalutazionepartecipantemeetingfisico"
  AFTER INSERT
  ON public.partecipantemeetingfisico
  FOR EACH ROW
  EXECUTE PROCEDURE public."Incrementa Valutazione Partecipanti Meeting"();
```

Trigger che richiama la funzione che controlla che il numero dei partecipanti ad un meeting fisico non superi il numero di posti della sala del meeting prima dell'inserimento del nuovo meeting.

```
CREATE FUNCTION public."Controllo NumeroMaxPartecipantiMeetingFisico"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
  numPostiDisponibili INTEGER;
  numDipendenti INTEGER;
BEGIN
  SELECT nPosti INTO numPostiDisponibili
  FROM MEETINGFISICO AS MF NATURAL JOIN SALAMEETING AS S
  WHERE MF.CodMF = NEW.CodMF;

  SELECT COUNT(CodF) INTO numDipendenti
  FROM PARTECIPANTEMEETINGFISICO AS PMF
  WHERE PMF.CodMF = NEW.CodMF
  GROUP BY PMF.CodMF;

  IF(numDipendenti > numPostiDisponibili) THEN
    RAISE EXCEPTION 'ATTENZIONE posti esauriti';
  END IF;

  RETURN NEW;
END;
$BODY$;

CREATE TRIGGER "Trigger_controllonumeromaxpartecipantimeetingfisico"
  BEFORE INSERT
  ON public.partecipantemeetingfisico
  FOR EACH ROW
  EXECUTE PROCEDURE public."Controllo NumeroMaxPartecipantiMeetingFisico"();
```

Trigger che richiama la funzione che controlla che non vengano aggiunti nuovi partecipanti ad un meeting fisico passato.

```
CREATE FUNCTION public."Controllo validitàdatainserimento partecipanti meeting fisic"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
    datacorrente DATE;
    datameeting DATE;
BEGIN
    SELECT datamf INTO datameeting
    FROM meetingfisico AS mf
    WHERE mf.codmf = NEW.codmf;

    datacorrente=CURRENT_DATE;

    IF(datacorrente > datameeting ) THEN
        RAISE EXCEPTION 'ATTENZIONE non puoi inserire dei partecipanti ad un meeting
passato';
    END IF;

    RETURN NEW;
END;
$BODY$;

CREATE TRIGGER "Trigger_validitàdatainserimentopartecipantimeetingfisico"
  BEFORE INSERT
  ON public.partecipantemeetingfisico
  FOR EACH ROW
  EXECUTE PROCEDURE public."Controllo validitàdatainserimento partecipanti meeting fisic"();
```

Trigger che richiama la funzione che controlla che il numero dei partecipanti ad un meeting telematico non superi il numero massimo di dispositivi collegabili della piattaforma del meeting prima dell'inserimento del nuovo meeting.

```
CREATE FUNCTION public."Controllo NumeroMaxPartecipantiMeetingTelematico"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
  numLimitePartecipanti INTEGER;
  numDipendenti INTEGER;
BEGIN
  SELECT nMaxDispositivi INTO numLimitePartecipanti
  FROM PIATTAFORMAMEETING AS PM NATURAL JOIN MEETINGTELEMATICO AS MT
  WHERE MT.CodMT = NEW.CodMT;

  SELECT COUNT(CodF) INTO numDipendenti
  FROM PARTECIPANTEMEETINGTELEMATICO AS PMT
  WHERE PMT.CodMT = NEW.CodMT
  GROUP BY PMT.CodMT;

  IF(numDipendenti > numLimitePartecipanti) THEN
    RAISE EXCEPTION 'ATTENZIONE superato numero massimo dispositivi ammessi ';

    END IF;

  RETURN NEW;
END;
$BODY$;

CREATE TRIGGER "Trigger_controllonumeromaxpartecipantimeetingtelematico"
  BEFORE INSERT
  ON public.partecipantemeetingtelematico
  FOR EACH ROW
  EXECUTE PROCEDURE public."Controllo NumeroMaxPartecipantiMeetingTelematico"();
```

Trigger che richiama la funzione per controllare che un partecipante non sia già occupato nell'arco temporale del meeting in cui è stato aggiunto prima dell'inserimento di un nuovo meeting Telematico.

```
CREATE FUNCTION
public."ControlloSovrapposizioneMeetingPerPartecipanteMeetingTelematico"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
  dataMeeting DATE;
  oraInizioMeeting TIME;
  oraFineMeeting TIME;
BEGIN
  SELECT MT.DataMT, MT.OraI, MT.OraF INTO dataMeeting, oraInizioMeeting, oraFineMeeting
  FROM PARTECIPANTEMEETINGTELEMATICO AS PMT NATURAL JOIN
  MEETINGTELEMATICO AS MT
  WHERE NEW.CodF = PMT.CodF AND NEW.CodMT = MT.CodMT;

  IF EXISTS (SELECT *
    FROM PARTECIPANTEMEETINGFISICO AS PMF NATURAL JOIN
    MEETINGFISICO AS MF
    WHERE NEW.CodF = PMF.CodF AND dataMeeting = MF.DataMF AND
      ((oraInizioMeeting BETWEEN MF.OraI AND MF.OraF) OR
      (oraFineMeeting BETWEEN MF.OraI AND MF.OraF)OR(MF.OraI BETWEEN
oraInizioMeeting AND oraFineMeeting)
      OR(MF.OraF BETWEEN oraInizioMeeting AND oraFineMeeting)))
    OR EXISTS (SELECT *
      FROM PARTECIPANTEMEETINGTELEMATICO AS PMT NATURAL JOIN
      MEETINGTELEMATICO AS MT
      WHERE NEW.CodF = PMT.CodF AND dataMeeting = MT.DataMT AND
        ((oraInizioMeeting BETWEEN MT.OraI AND MT.OraF) OR
        (oraFineMeeting BETWEEN MT.OraI AND MT.OraF)OR(MT.OraI
BETWEEN oraInizioMeeting AND oraFineMeeting)
        OR(MT.OraF BETWEEN oraInizioMeeting AND oraFineMeeting)))
  THEN
    RAISE EXCEPTION 'In quelle ore questo dipendente partecipa già ad un altro meeting';
  END IF;

  RETURN NEW;
END;
$BODY$;

CREATE TRIGGER "Trigger_controllosovrapposizionemeetingperpartecipantemeetingte"
  BEFORE INSERT
  ON public.partecipantemeetingtelematico
  FOR EACH ROW
  EXECUTE PROCEDURE
public."ControlloSovrapposizioneMeetingPerPartecipanteMeetingTelematico"();
```

Trigger che richiama la funzione che controlla che non vengano aggiunti nuovi partecipanti ad un meeting telematico passato.

```
CREATE FUNCTION public."Controllo validitàdatainserimento partecipanti meeting telemat"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
    datacorrente DATE;
    datameeting DATE;
BEGIN
    SELECT datamt INTO datameeting
    FROM meetingtelematico AS mt
    WHERE mt.codmt = NEW.codmt;

    datacorrente=CURRENT_DATE;

    IF(datacorrente > datameeting ) THEN
        RAISE EXCEPTION 'ATTENZIONE non puoi inserire dei partecipanti ad un meeting
passato';
    END IF;

    RETURN NEW;
END;
$BODY$;
```

```
CREATE TRIGGER "Trigger_controllovaliditàdatainserimentopartecipantimeetingtel"
  BEFORE INSERT
  ON public.participantemeetingtelematico
  FOR EACH ROW
  EXECUTE PROCEDURE public."Controllo validitàdatainserimento partecipanti meeting
telemat"();
```

```
CREATE FUNCTION public."Decrementa Valutazione Partecipanti Meeting Eliminati"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
    valutazioneAggiornata REAL;
BEGIN
    SELECT valutazione INTO valutazioneAggiornata
    FROM DIPENDENTE
    WHERE CodF = OLD.CodF;

    valutazioneAggiornata := valutazioneAggiornata - 0.5;

    UPDATE DIPENDENTE
    SET valutazione = valutazioneAggiornata
    WHERE CodF = OLD.CodF;
```

```
RETURN NULL;
END;
$BODY$;
```

Trigger che richiama la funzione che decrementa la valutazione di un partecipante al meeting Telematico dopo la sua eliminazione.

```
CREATE TRIGGER "Trigger_decrementavalutazionepartecipantemeetingtelematicoelemi"
AFTER DELETE
ON public.partecipantemeetingtelematico
FOR EACH ROW
EXECUTE PROCEDURE public."Decrementa Valutazione Partecipanti Meeting Eliminati"();
```

Trigger che richiama la funzione che incrementa la valutazione di un partecipante al meeting dopo essere stato aggiunto ad un meeting Telematico.

```
CREATE FUNCTION public."Incrementa Valutazione Partecipanti Meeting"()
RETURNS trigger
LANGUAGE 'plpgsql'
COST 100
VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
    valutazioneAggiornata REAL;
BEGIN
    SELECT valutazione INTO valutazioneAggiornata
    FROM DIPENDENTE
    WHERE CodF = NEW.CodF;

    valutazioneAggiornata := valutazioneAggiornata + 0.5;

    UPDATE DIPENDENTE
    SET valutazione = valutazioneAggiornata
    WHERE CodF = NEW.CodF;

    RETURN NULL;
END;
$BODY$;
```

```
CREATE TRIGGER "Trigger_incrementovalutazionepartecipantemeetingtelematico"
AFTER INSERT
ON public.partecipantemeetingtelematico
FOR EACH ROW
EXECUTE PROCEDURE public."Incrementa Valutazione Partecipanti Meeting"();
```


Trigger che richiama la funzione che controlla che ci sia un project manager dopo l'inserimento di un nuovo progetto.

```
CREATE FUNCTION public."Controllo Presenza Project Manager"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
BEGIN
  IF NOT EXISTS (SELECT *
    FROM PARTECIPANTEPROGETTO AS P
    WHERE P.CodP = NEW.CodP AND P.Ruolo LIKE 'PROJECT MANAGER') THEN
    RAISE EXCEPTION 'ATTENZIONE Il progetto deve avere un Project Manager';
  END IF;

  RETURN NEW;
END;
$BODY$;

CREATE TRIGGER "Trigger_controllopresenzaprojectmanager"
  AFTER INSERT
  ON public.partecipanteprogetto
  FOR EACH ROW
  EXECUTE PROCEDURE public."Controllo Presenza Project Manager"();
```

Trigger che richiama la funzione che blocca l'inserimento di nuovi partecipanti ad un progetto finito.

```
CREATE FUNCTION public."Controllo validitàdatainserimento partecipanti progetto"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
    statusprogetto VARCHAR(10);
BEGIN
    SELECT status INTO statusprogetto
    FROM PROGETTO AS PR
    WHERE PR.codP = NEW.codP;

    IF(statusprogetto LIKE 'FINITO' ) THEN
        RAISE EXCEPTION 'ATTENZIONE non puoi inserire dei partecipanti ad un progetto
finito';
    END IF;

    RETURN NEW;
END;
$BODY$;

CREATE TRIGGER "Trigger_controllovaliditàdatainserimentopartecipantiprogetto"
  BEFORE INSERT
  ON public.partecipanteprogetto
  FOR EACH ROW
  EXECUTE PROCEDURE public."Controllo validitàdatainserimento partecipanti progetto"();
```

Trigger che richiama la funzione che decrementa la valutazione dei partecipanti dopo l'eliminazione dei partecipanti che non siano un project manager da un progetto.

```
CREATE FUNCTION public."Decrementa Valutazione Partecipanti Progetto Eliminati"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
  valutazioneAggiornata REAL;
BEGIN
  SELECT valutazione INTO valutazioneAggiornata
  FROM DIPENDENTE
  WHERE CodF = OLD.CodF;

  valutazioneAggiornata := valutazioneAggiornata - 1;

  UPDATE DIPENDENTE
  SET valutazione = valutazioneAggiornata
  WHERE CodF = OLD.CodF;

  RETURN NULL;
END;
$BODY$;

CREATE TRIGGER "Trigger_decrementavalutazionepartecipantiprogettoeliminati"
  AFTER DELETE
  ON public.partecipanteprogetto
  FOR EACH ROW
  WHEN (old.ruolo::text !~ 'PROJECT MANAGER'::text)
  EXECUTE PROCEDURE public."Decrementa Valutazione Partecipanti Progetto Eliminati"();
```

Trigger che richiama la funzione che decrementa la valutazione del project manager dopo l'eliminazione di esso da un progetto.

```
CREATE FUNCTION public."Decrementa Valutazione Project Manager Eliminato"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
  valutazioneAggiornata REAL;
BEGIN
  SELECT valutazione INTO valutazioneAggiornata
  FROM DIPENDENTE
  WHERE CodF = OLD.CodF;

  valutazioneAggiornata := valutazioneAggiornata - 2;

  UPDATE DIPENDENTE
  SET valutazione = valutazioneAggiornata
  WHERE CodF = OLD.CodF;

  RETURN NULL;
END;
$BODY$;

CREATE TRIGGER "Trigger_decrementavalutazioneprojectmanagereliminato"
  AFTER DELETE
  ON public.partecipanteprogetto
  FOR EACH ROW
  WHEN (old.ruolo::text ~~ 'PROJECT MANAGER'::text)
  EXECUTE PROCEDURE public."Decrementa Valutazione Project Manager Eliminato"();
```

Trigger che richiama la funzione che incrementa la valutazione dei nuovi partecipanti dopo l'aggiunta di nuovi partecipanti che non siano un project manager ad un progetto.

```
CREATE FUNCTION public."Incrementa Valutazione Partecipanti Progetto"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
  valutazioneAggiornata REAL;
BEGIN
  SELECT valutazione INTO valutazioneAggiornata
  FROM DIPENDENTE
  WHERE CodF = NEW.CodF;

  valutazioneAggiornata := valutazioneAggiornata + 1;

  UPDATE DIPENDENTE
  SET valutazione = valutazioneAggiornata
  WHERE CodF = NEW.CodF;

  RETURN NULL;
END;
$BODY$;

CREATE TRIGGER "Trigger_incrementovalutazionepartecipantiprogetto"
  AFTER INSERT
  ON public.partecipanteprogetto
  FOR EACH ROW
  WHEN (new.ruolo::text !~~ 'PROJECT MANAGER'::text)
  EXECUTE PROCEDURE public."Incrementa Valutazione Partecipanti Progetto"();
```

Trigger che richiama la funzione che incrementa la valutazione del nuovo project manager dopo l'aggiunta di ad un progetto.

```
CREATE FUNCTION public."Incrementa Valutazione Project Manager"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
  valutazioneAggiornata REAL;
BEGIN
  SELECT valutazione INTO valutazioneAggiornata
  FROM DIPENDENTE
  WHERE CodF = NEW.CodF;

  valutazioneAggiornata := valutazioneAggiornata + 2;

  UPDATE DIPENDENTE
  SET valutazione = valutazioneAggiornata
  WHERE CodF = NEW.CodF;

  RETURN NULL;
END;
$BODY$;

CREATE TRIGGER "Trigger_incrementovalutazioneprojectmanager"
  AFTER INSERT
  ON public.partecipanteprogetto
  FOR EACH ROW
  WHEN (new.ruolo::text ~~ 'PROJECT MANAGER'::text)
  EXECUTE PROCEDURE public."Incrementa Valutazione Project Manager"();
```

Trigger che richiama la funzione che controlla che ci sia un solo project manager prima dell'inserimento di un nuovo progetto.

```
CREATE FUNCTION public."Unicità Project Manager"()
  RETURNS trigger
  LANGUAGE 'plpgsql'
  COST 100
  VOLATILE NOT LEAKPROOF
AS $BODY$
BEGIN
  IF EXISTS (SELECT *
             FROM PARTECIPANTEPROGETTO AS P
             WHERE P.CodP = NEW.CodP AND P.Ruolo LIKE 'PROJECT MANAGER') THEN
    RAISE EXCEPTION 'ATTENZIONE non puoi inserire un altro project manager';
  END IF;

  RETURN NEW;
END;
$BODY$;
```

```
CREATE TRIGGER "Trigger_unicitàprojectmanager"
  BEFORE INSERT
  ON public.partecipanteprogetto
  FOR EACH ROW
  WHEN (new.ruolo::text ~~ 'PROJECT MANAGER'::text)
  EXECUTE PROCEDURE public."Unicità Project Manager"();
```

6 Popolamento con Dati d'Esempio

--Dipendenti e rispettive residenze

```
INSERT INTO DIPENDENTE
(CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO)
VALUES
('PNCGPT00M25F839B','GIANPIETRO','PANICO','GIANPIETRO@GMAIL.COM',TO_DATE('25/08/2000','DD/MM/YYYY'),'M','393334554321','/fotoDipendenti/fotoprofilodefault2.jpg',10000.00)

INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES
('PNCGPT00M25F839B','NA','80121','GIOSUE' CARDUCCI','12','NAPOLI')

INSERT INTO DIPENDENTE
(CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO)
VALUES
('GRDSTF99D05E791R','STEFANO','GIORDANO','STEFANOG@GMAIL.COM',TO_DATE('05/08/1999','DD/MM/YYYY'),'M','393667899076','/fotoDipendenti/fotoprofilodefault2.jpg',1500.00);

INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES
('GRDSTF99D05E791R','CE','81020','CORSO I OTTOBRE','20','CASERTA');

INSERT INTO DIPENDENTE
(CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO)
VALUES
('VNTDMN98D31E791P','DOMENICO','VENTRIGLIA','VENTRIGLIAD@GMAIL.COM',TO_DATE('31/08/1998','DD/MM/YYYY'),'M','393662676548','/fotoDipendenti/fotoprofilodefault2.jpg',1200.00);

INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES
('VNTDMN98D31E791P','CE','81024','CORNATO','120','MADDALONI');
INSERT INTO DIPENDENTE
(CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO)
VALUES
('CRRCLD98P27E791Q','CLAUDIA','CERRETO','CLAUDIA C98@GMAIL.COM',TO_DATE('27/09/1998','DD/MM/YYYY'),'F','393396627809','/fotoDipendenti/fotoprofilodefault2.jpg',9000.00);

INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES
('CRRCLD98P27E791Q','CE','81024','MONTELLA','3/B','MADDALONI');

INSERT INTO DIPENDENTE
(CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO)
VALUES ('DNZMRC98P27E791T','MARCELLA','DI
NUZZO','DINUZZOMARCELLA@GMAIL.COM',TO_DATE('27/09/1998','DD/MM/YYYY'),'F','39388728838','/fotoDipendenti/fotoprofilodefault2.jpg',2000.00);

INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES
('DNZMRC98P27E791T','CE','81021','VIA ROMA','43','SAN NICOLA');

INSERT INTO DIPENDENTE
(CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO)
VALUES
```



```
('NZZDVD93A13E791D','DAVIDE','NUZZO','DAVIDENUZZO@GMAIL.COM',TO_DATE('13/01/1993','DD/MM/YYYY'),'M','393364558721','/fotoDipendenti/fotoprofilodefault2.jpg',1400.00);
```

```
INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES ('NZZDVD93A13E791D','CE','81024','SFRANCESCO','12','MADDALONI');
```

```
INSERT INTO DIPENDENTE (CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO) VALUES ('SFRSNA91G09E791U','SONIA','SFERRAGATTA','SONIAS@GMAIL.COM',TO_DATE('09/07/1991','DD/MM/YYYY'),'F','393331243658','/fotoDipendenti/fotoprofilodefault2.jpg',1600.00);
```

```
INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES ('SFRSNA91G09E791U','LU','55049','LUIGI SCALFARO','15','TORRE DEL LAGO');
```

```
INSERT INTO DIPENDENTE (CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO) VALUES ('SFRTRS89E03E791F','TERESA','SFERRAGATTA','TERESAS@GMAIL.COM',TO_DATE('03/05/1989','DD/MM/YYYY'),'F','393312578908','/fotoDipendenti/fotoprofilodefault2.jpg',1700.00);
```

```
INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES ('SFRTRS89E03E791F','LU','55050','LUCCA','12','MASSAROSA');
```

```
INSERT INTO DIPENDENTE (CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO) VALUES ('DDTGLO98S21E791M','GIULIO','DIODATI','GIULIO@GMAIL.COM',TO_DATE('21/12/1998','DD/MM/YYYY'),'M','391407898345','/fotoDipendenti/fotoprofilodefault2.jpg',1200.00);
```

```
INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES ('DDTGLO98S21E791M','CE','81020','TRIESTE','12','CASERTA');
```

```
INSERT INTO DIPENDENTE (CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO) VALUES ('CRSGNM00F84G782G','GIANMARCO','CARUSO','GIANMARCO@GMAIL.COM',TO_DATE('05/07/2000','DD/MM/YYYY'),'M','393354785477','/fotoDipendenti/fotoprofilodefault2.jpg',5000.00);
```

```
INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES ('CRSGNM00F84G782G','NA','80125','POSILLIPO','4','NAPOLI');
```

```
INSERT INTO DIPENDENTE (CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO) VALUES ('MTTPOL00H78F964B','PAOLO','MOTTA','PAOLOM@GMAIL.COM',TO_DATE('30/03/1999','DD/MM/YYYY'),'M','39336587455','/fotoDipendenti/fotoprofilodefault2.jpg',6000.00);
```

```
INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES ('MTTPOL00H78F964B','NA','80125','POSILLIPO','43','NAPOLI');
```

```

INSERT INTO DIPENDENTE
(CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO)
VALUES
('CNTVTT00M89N765D','VITTORIO','CENTOFANTI','VITTOR@GMAIL.COM',TO_DATE('30/08/
1990','DD/MM/YYYY'),'M','393354774100','/fotoDipendenti/fotoprofilodefault2.jpg','2000.00');

INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES
('CNTVTT00M89N765D','NA','80122','MANZONI','50','NAPOLI')

INSERT INTO DIPENDENTE
(CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO)
VALUES
('NBLADR00M78E430N','ANDREA','ANNIBALLO','ANREANN@GMAIL.COM',TO_DATE('20/12/
1995','DD/MM/YYYY'),'M','39336558710','/fotoDipendenti/fotoprofilodefault2.jpg','3000.00');

INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES
('NBLADR00M78E430N','MI','20019','DUOMO','98','MILANO')

INSERT INTO DIPENDENTE
(CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO)
VALUES
('CNLCML00F84V962U','CAMILLA','CONCILIO','CAMILLACONC@GMAIL.COM',TO_DATE('05/1
1/1998','DD/MM/YYYY'),'F','39334588574','/fotoDipendenti/fotoprofilodefault2.jpg','2500.00');

INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES
('CNLCML00F84V962U','NA','80122','PETRARCA','98','NAPOLI')

INSERT INTO DIPENDENTE
(CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO)
VALUES
('PCTEML00H98F012G','EMILIO','PUCCETTONE','EMILIOPUCC@GMAIL.COM',TO_DATE('01/1
1/1992','DD/MM/YYYY'),'M','393339987450','/fotoDipendenti/fotoprofilodefault2.jpg','1500.00');

INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES
('PCTEML00H98F012G','NA','80122','NAPOLI','76','NAPOLI')

INSERT INTO DIPENDENTE
(CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO)
VALUES
('CMPMRA00K85D893G','MARIA','CAMPANILE','MARIAC@GMAIL.COM',TO_DATE('20/04/199
7','DD/MM/YYYY'),'F','39336587558','/fotoDipendenti/fotoprofilodefault2.jpg','1500.00');

INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES
('CMPMRA00K85D893G','MI','20021','FERNANDES','76','MILANO')

INSERT INTO DIPENDENTE
(CODF,NOME,COGNOME,EMAIL,DATAN,SESSO,NUMCELLULARE,PATHFOTO,SALARIO)
VALUES
('TDSUGO00M29G587G','UGO','TEDESCO','UGOT@GMAIL.COM',TO_DATE('20/04/1996','DD/
MM/YYYY'),'M','39336588003','/fotoDipendenti/fotoprofilodefault2.jpg','4000.00');

INSERT INTO RESIDENZA (CODF,PROVINCIA,CAP,VIA,NCIVICO,città) VALUES
('TDSUGO00M29G587G','TO','10121','ROMA','34','TORINO')

```

--Inserimento sale

```
INSERT INTO SALAMEETING VALUES ('AULA MAGNA',150),('A1',50);
```

--Inserimento piattaforme

```
INSERT INTO PIATTAFORMAMEETING VALUES ('MICROSOFT TEAMS',150),('GOOGLE MEET',50);
```

--Inserimento progetti con rispettivi ambiti e partecipanti

```
INSERT INTO PROGETTO VALUES  
(DEFAULT,'PGD','VIDEOGAME',TO_DATE('20/01/2020','DD/MM/YYYY'),  
TO_DATE('01/07/2021','DD/MM/YYYY'),'NINTENDO','SVILUPPO DI UN VIDEOGIOCO  
PLATFORM');
```

```
INSERT INTO AMBITO VALUES(CURRVAL('CODP'),'GAMEDESIGN');
```

```
INSERT INTO AMBITO VALUES(CURRVAL('CODP'),'PLATFORM GAME');
```

```
INSERT INTO AMBITO VALUES(CURRVAL('CODP'),'GAMEDEVELOPMENT');
```

```
INSERT INTO PARTECIPANTEPROGETTO('SCTGDP41V19E791U',CURRVAL('CODP'),'PROJECT  
MANAGER');
```

```
INSERT INTO PARTECIPANTEPROGETTO('TGLNGL96R04E791R',CURRVAL('CODP'),'GAME  
DEVELOPER');
```

```
INSERT INTO PARTECIPANTEPROGETTO('CRRCLD98P27E791Q',CURRVAL('CODP'),'GAME  
DESIGNER');
```

```
INSERT INTO PROGETTO VALUES  
(DEFAULT,'UNIVERSITA','UNIVERSITARIO',TO_DATE('06/02/2021','DD/MM/YYYY'),  
TO_DATE('07/03/2021','DD/MM/YYYY'),'CORSO OOB'D','CORSI DI OBJECT ORIENTATION E  
BASI DI DATI I');
```

```
INSERT INTO AMBITO VALUES(CURRVAL('CODP'),'INFORMATICO');
```

```
INSERT INTO AMBITO VALUES(CURRVAL('CODP'),'JAVA');
```

```
INSERT INTO AMBITO VALUES(CURRVAL('CODP'),'DATABASE');
```

```
INSERT INTO  
PARTECIPANTEPROGETTO('PNCGPT00M25F839B',CURRVAL('CODP'),'PROJECT MANAGER');
```

```
INSERT INTO PROGETTO VALUES (DEFAULT,'Ricerca  
Ambientale','Ambientale',TO_DATE('01/04/2020','DD/MM/YYYY'),  
TO_DATE('31/03/2024','DD/MM/YYYY'),'WWF','SISTEMA PER IL MONITORAGGIO DI FLORA  
E FAUNA NEI PRESSI DI ZONE AD ALTO RISCHIO INQUINAMENTO');
```

```
INSERT INTO AMBITO VALUES(CURRVAL('CODP'),'ECOLOGIA');
```

```
INSERT INTO AMBITO VALUES(CURRVAL('CODP'),'ZOOLOGIA');
```

INSERT INTO AMBITO VALUES(CURRVAL('CODP'),'BOTANICA');

INSERT INTO
PARTECIPANTEPROGETTO('CNTVTTT0oM89N765D',CURRVAL('CODP'),'PROJECT
MANAGER');

INSERT INTO
PARTECIPANTEPROGETTO('MTTPOL0oH78F964B',CURRVAL('CODP'),'SVILUPPO
SOFTWARE');

INSERT INTO
PARTECIPANTEPROGETTO('CRSGNM0oF84G782G',CURRVAL('CODP'),'SVILUPPO SENSORI');

INSERT INTO
PARTECIPANTEPROGETTO('CNLCML0oF84V962U',CURRVAL('CODP'),'RICERCATRICE');

--Inserimento meeting fisici e rispettivi partecipanti

INSERT INTO MEETINGFISICO VALUES (DEFAULT,
TO_DATE('29/03/2021','DD/MM/YYYY'),'21:03:00','23:03:00','AULAMAGNA');

INSERT INTO PARTECIPANTEMEETINGFISICO('CNTVTTT0oM89N765D', CURRVAL('CODMF'));

INSERT INTO PARTECIPANTEMEETINGFISICO('DDTGLO98S21E791M', CURRVAL('CODMF'));

INSERT INTO PARTECIPANTEMEETINGFISICO('NZZDVD93A13E791D', CURRVAL('CODMF'));

INSERT INTO PARTECIPANTEMEETINGFISICO('CRSGNM0oF84G782G',
CURRVAL('CODMF'));

INSERT INTO MEETINGFISICO VALUES (DEFAULT,
TO_DATE('22/03/2021','DD/MM/YYYY'),'21:22:00','23:22:00','AULAMAGNA');

INSERT INTO PARTECIPANTEMEETINGFISICO('PNCGPT0oM25F839B', CURRVAL('CODMF'));

INSERT INTO PARTECIPANTEMEETINGFISICO('SFRTRS89E03E791F', CURRVAL('CODMF'));

INSERT INTO MEETINGFISICO VALUES (DEFAULT,
TO_DATE('05/03/2021','DD/MM/YYYY'),'21:25:00','22:25:00','AULAMAGNA');

INSERT INTO PARTECIPANTEMEETINGFISICO('CRSGNM0oF84G782G',
CURRVAL('CODMF'));

INSERT INTO PARTECIPANTEMEETINGFISICO('SFRSNA91G09E791U', CURRVAL('CODMF'));

INSERT INTO PARTECIPANTEMEETINGFISICO('MTTPOL0oH78F964B', CURRVAL('CODMF'));

INSERT INTO PARTECIPANTEMEETINGFISICO('DDTGLO98S21E791M', CURRVAL('CODMF'));

INSERT INTO PARTECIPANTEMEETINGFISICO('PCTEML0oH98F012G', CURRVAL('CODMF'));

--Inserimento meeting telematici e rispettivi partecipanti

```
INSERT INTO MEETINGTELEMATICO VALUES (DEFAULT,  
TO_DATE('23/06/2021','DD/MM/YYYY'),'22:38:00','23:38:00','MICROSOFT TEAMS');
```

```
INSERT INTO PARTECIPANTEMEETINGFISICO('NVLADRooM78E43oN',  
CURRVAL('CODMT'));
```

```
INSERT INTO PARTECIPANTEMEETINGFISICO('TDSUGOooM29G587G',  
CURRVAL('CODMT'));
```

```
INSERT INTO PARTECIPANTEMEETINGFISICO('VNTDMN98D31E791P', CURRVAL('CODMT'));
```

Gruppo 15 Traccia 2

Gianpietro Panico N86003500

Antonio Tagliafierro N86003148